

```
x = 0
e = 1
for a in range(1,21):
    e += 5
    e = e+a
    print(f"el valor de a es igual a: {a} y el valor de e es igual a: {e}")
```

```
el valor de a es igual a: 1 y el valor de e es igual a: 7
el valor de a es igual a: 2 y el valor de e es igual a: 14
el valor de a es igual a: 3 y el valor de e es igual a: 22
el valor de a es igual a: 4 y el valor de e es igual a: 31
el valor de a es igual a: 5 y el valor de e es igual a: 41
el valor de a es igual a: 6 y el valor de e es igual a: 52
el valor de a es igual a: 7 y el valor de e es igual a: 64
el valor de a es igual a: 8 y el valor de e es igual a: 77
el valor de a es igual a: 9 y el valor de e es igual a: 91
el valor de a es igual a: 10 y el valor de e es igual a: 106
el valor de a es igual a: 11 y el valor de e es igual a: 122
el valor de a es igual a: 12 y el valor de e es igual a: 139
el valor de a es igual a: 13 y el valor de e es igual a: 157
el valor de a es igual a: 14 y el valor de e es igual a: 176
el valor de a es igual a: 15 y el valor de e es igual a: 196
el valor de a es igual a: 16 y el valor de e es igual a: 217
el valor de a es igual a: 17 y el valor de e es igual a: 239
el valor de a es igual a: 18 y el valor de e es igual a: 262
el valor de a es igual a: 19 y el valor de e es igual a: 286
el valor de a es igual a: 20 y el valor de e es igual a: 311
```

```
print(f"ingresa un valor real y entero:")
x = int(input("ingresa el entero:"))

for a in range(10,31):
    y = x + 2
    z = y + 1
    x += 1
    y1 = y+x
    # print(f"los valores son: {[x,a,y,z,x,y1]}")
    print(f"los valores son: [x={x},a={a},y={y},z={z},x={x},y1={y1}]")

print(f"y1 es igual a {y1}")
```

```
ingresa un valor real y entero:
ingresa el entero:2
los valores son: [x=3,a=10,y=4,z=5,x=3,y1=7]
los valores son: [x=4,a=11,y=5,z=6,x=4,y1=9]
los valores son: [x=5,a=12,y=6,z=7,x=5,y1=11]
los valores son: [x=6,a=13,y=7,z=8,x=6,y1=13]
los valores son: [x=7,a=14,y=8,z=9,x=7,y1=15]
los valores son: [x=8,a=15,y=9,z=10,x=8,y1=17]
los valores son: [x=9,a=16,y=10,z=11,x=9,y1=19]
los valores son: [x=10,a=17,y=11,z=12,x=10,y1=21]
los valores son: [x=11,a=18,y=12,z=13,x=11,y1=23]
los valores son: [x=12,a=19,y=13,z=14,x=12,y1=25]
los valores son: [x=13,a=20,y=14,z=15,x=13,y1=27]
los valores son: [x=14,a=21,y=15,z=16,x=14,y1=29]
los valores son: [x=15,a=22,y=16,z=17,x=15,y1=31]
los valores son: [x=16,a=23,y=17,z=18,x=16,y1=33]
los valores son: [x=17,a=24,y=18,z=19,x=17,y1=35]
los valores son: [x=18,a=25,y=19,z=20,x=18,y1=37]
los valores son: [x=19,a=26,y=20,z=21,x=19,y1=39]
los valores son: [x=20,a=27,y=21,z=22,x=20,y1=41]
los valores son: [x=21,a=28,y=22,z=23,x=21,y1=43]
los valores son: [x=22,a=29,y=23,z=24,x=22,y1=45]
los valores son: [x=23,a=30,y=24,z=25,x=23,y1=47]
y1 es igual a 47
```

```
n = 0
for a in range(1,16):
    x = 2*n+1
    n += 1
    print(f"x={x} y n={n}")
```

```
x=1 y n=1
x=3 y n=2
x=5 y n=3
x=7 y n=4
x=9 y n=5
x=11 y n=6
x=13 y n=7
x=15 y n=8
x=17 y n=9
x=19 y n=10
x=21 y n=11
x=23 y n=12
x=25 y n=13
x=27 y n=14
x=29 y n=15
```

```
import numpy as np

arreglo = np . array([10,20,30,40,50])
suma = 0
for valor in arreglo :
    suma += valor

print(f"la suma del arreglo es: {suma}")
```

la suma del arreglo es: 150

```
a = range(1,21)

arreglo = a
suma = 0
for valor in arreglo :
    suma += valor

print(f"la suma del arreglo es: {suma}")
```

la suma del arreglo es: 210

```
matriz = [
    [1 ,2, 3] ,
    [4, 5, 6] ,
    [7, 8, 9]
]

for fila in matriz :
    for elemento in fila :
        print(elemento, end = "-")
    print()
```

1-2-3-
4-5-6-
7-8-9-

Propiedades de las Sumatorias para la Integral Definida

Resumen Matemático

September 3, 2026

Introducción

La integral definida de una función $f(x)$ en un intervalo $[a, b]$ se define mediante el límite de una suma de Riemann:

$$\int_a^b f(x) dx = \lim_{n \rightarrow \infty} \sum_{i=1}^n f(x_i^*) \Delta x$$

donde $\Delta x = \frac{b-a}{n}$ y x_i^* es un punto muestra en el subintervalo i -ésimo. Para poder calcular este límite de manera eficiente, es necesario dominar las propiedades algebraicas de las sumatorias y conocer las sumas notables.

1 Propiedades Básicas de las Sumatorias

Sean $\{a_i\}$ y $\{b_i\}$ sucesiones de números reales, y sea c una constante. Entonces:

1. **Linealidad:**

$$\begin{aligned} \sum_{i=1}^n (a_i + b_i) &= \sum_{i=1}^n a_i + \sum_{i=1}^n b_i \\ \sum_{i=1}^n c \cdot a_i &= c \sum_{i=1}^n a_i \end{aligned}$$

2. **Suma de una constante:**

$$\sum_{i=1}^n c = c \cdot n$$

3. **Propiedad telescópica:**

$$\sum_{i=1}^n (a_i - a_{i-1}) = a_n - a_0$$

(Útil para simplificar sumas cuando los términos se cancelan).

4. **Separación de términos:**

$$\sum_{i=1}^n a_i = \sum_{i=1}^m a_i + \sum_{i=m+1}^n a_i \quad \text{para } 1 \leq m < n$$

2 Sumas Notables (Fundamentales para Límites)

Cuando $n \rightarrow \infty$, aparecen expresiones polinómicas en n . Estas fórmulas son imprescindibles:

1. Suma de los primeros n naturales:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

2. Suma de los cuadrados:

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

3. Suma de los cubos:

$$\sum_{i=1}^n i^3 = \left[\frac{n(n+1)}{2} \right]^2$$

4. Suma de potencias superiores (generalización):

$$\sum_{i=1}^n i^k = \frac{n^{k+1}}{k+1} + O(n^k) \quad (\text{para } k \geq 1)$$

Esto significa que el término dominante es $\frac{n^{k+1}}{k+1}$, lo cual es clave al tomar límites.

3 Conexión con la Integral Definida

Al calcular la integral mediante límites, se usa la suma de Riemann por la derecha (o izquierda). Por ejemplo, para $f(x) = x^p$ en $[0, 1]$:

$$\int_0^1 x^p dx = \lim_{n \rightarrow \infty} \sum_{i=1}^n \left(\frac{i}{n} \right)^p \cdot \frac{1}{n} = \lim_{n \rightarrow \infty} \frac{1}{n^{p+1}} \sum_{i=1}^n i^p$$

Usando la suma notable correspondiente:

$$= \lim_{n \rightarrow \infty} \frac{1}{n^{p+1}} \left(\frac{n^{p+1}}{p+1} + \text{términos de menor orden} \right) = \frac{1}{p+1}$$

Este proceso muestra cómo las propiedades de las sumatorias permiten evaluar el límite sin dificultad.

4 Ejemplo Completo

Calcular $\int_0^1 x^2 dx$ usando la definición:

$$\begin{aligned}
\int_0^1 x^2 dx &= \lim_{n \rightarrow \infty} \sum_{i=1}^n \left(\frac{i}{n} \right)^2 \cdot \frac{1}{n} \\
&= \lim_{n \rightarrow \infty} \frac{1}{n^3} \sum_{i=1}^n i^2 \\
&= \lim_{n \rightarrow \infty} \frac{1}{n^3} \cdot \frac{n(n+1)(2n+1)}{6} \\
&= \lim_{n \rightarrow \infty} \frac{2n^3 + 3n^2 + n}{6n^3} \\
&= \lim_{n \rightarrow \infty} \left(\frac{2}{6} + \frac{3}{6n} + \frac{1}{6n^2} \right) = \frac{1}{3}
\end{aligned}$$

Conclusión

Las propiedades de linealidad y las sumas notables son herramientas algebraicas que transforman el límite de una suma de Riemann en un cálculo sencillo. Dominar estas identidades es esencial para comprender la definición formal de integral definida y su relación con el área bajo una curva.

Uso del Ciclo `for` en Python

Documento Explicativo

3 de septiembre de 2026

Índice

1. Introducción	2
2. Ciclo <code>for</code> con Listas	2
2.1. Ejemplo: Suma de elementos en una lista	2
2.2. Ejemplo: Modificar elementos de una lista	2
3. Ciclo <code>for</code> con Arreglos (Arrays)	2
3.1. Ejemplo: Arreglo unidimensional	2
3.2. Ejemplo: Arreglo bidimensional con índices	3
4. Ciclo <code>for</code> con Matrices	3
4.1. Ejemplo: Operaciones con matrices	3
4.2. Ejemplo: Multiplicación de matrices	3
5. Ciclo <code>for</code> con Tuplas	4
5.1. Ejemplo: Recorriendo una tupla	4
5.2. Ejemplo: Tupla de tuplas (matriz inmutable)	4
5.3. Ejemplo: Desempaquetado en tuplas	5
6. Ejercicios Prácticos	5
6.1. Ejercicio 1: Procesamiento de listas	5
6.2. Ejercicio 2: Diagonal de una matriz	5
6.3. Ejercicio 3: Operaciones con tuplas	5
7. Conclusión	6

1. Introducción

El ciclo `for` es una estructura de control fundamental en Python que permite iterar sobre elementos de una secuencia. En este documento exploraremos su uso con diferentes estructuras de datos: listas, arreglos (arrays), matrices y tuplas.

2. Ciclo for con Listas

Las listas son estructuras de datos mutables que pueden contener elementos de diferentes tipos. El ciclo `for` nos permite recorrer cada elemento de la lista.

2.1. Ejemplo: Suma de elementos en una lista

```
1 # Definimos una lista de numeros
2 numeros = [10, 20, 30, 40, 50]
3
4 # Variable para almacenar la suma
5 suma_total = 0
6
7 # Recorremos cada elemento de la lista
8 for numero in numeros:
9     suma_total += numero
10    print(f"Sumando {numero}, suma actual: {suma_total}")
11
12 print(f"\nLa suma total es: {suma_total}")
```

Listing 1: Suma de elementos de una lista

2.2. Ejemplo: Modificar elementos de una lista

```
1 # Lista de nombres
2 nombres = ["ana", "luis", "maria", "jose"]
3
4 # Convertir cada nombre a may sculas
5 for i in range(len(nombres)):
6     nombres[i] = nombres[i].upper()
7
8 print("Nombres en may sculas:", nombres)
```

Listing 2: Modificando elementos de una lista

3. Ciclo for con Arreglos (Arrays)

En Python, los arreglos se pueden implementar usando el módulo `array` o más comúnmente usando NumPy. Aquí usaremos listas para simular arreglos unidimensionales y bidimensionales.

3.1. Ejemplo: Arreglo unidimensional

```
1 # Arreglo de temperaturas
2 temperaturas = [23.5, 24.0, 22.8, 25.1, 26.3]
3
4 # Calcular el promedio
5 suma_temp = 0
6 for temp in temperaturas:
7     suma_temp += temp
8
9 promedio = suma_temp / len(temperaturas)
10 print(f"Temperaturas: {temperaturas}")
11 print(f"Temperatura promedio: {promedio:.2f} C ")
```

Listing 3: Recorriendo un arreglo unidimensional

3.2. Ejemplo: Arreglo bidimensional con índices

```
1 # Arreglo bidimensional (matriz) de calificaciones
2 # Filas: estudiantes, Columnas: materias
3 calificaciones = [
4     [85, 90, 78],
5     [92, 88, 95],
6     [70, 75, 80],
7     [88, 92, 85]
8 ]
9
10 # Mostrar cada calificación con su posición
11 print("Calificaciones de estudiantes:")
12 for i in range(len(calificaciones)):
13     for j in range(len(calificaciones[i])):
14         print(f"Estudiante {i+1}, Materia {j+1}: {calificaciones[i][j]}")
```

Listing 4: Recorriendo un arreglo bidimensional

4. Ciclo for con Matrices

Las matrices son arreglos bidimensionales que pueden ser representados como listas de listas en Python.

4.1. Ejemplo: Operaciones con matrices

```
1 # Definimos dos matrices 2x2
2 matriz1 = [
3     [1, 2],
4     [3, 4]
5 ]
6
7 matriz2 = [
8     [5, 6],
9     [7, 8]
10 ]
11
12 # Crear matriz resultado inicializada con ceros
13 resultado = [
14     [0, 0],
15     [0, 0]
16 ]
17
18 # Sumar las matrices
19 for i in range(len(matriz1)):
20     for j in range(len(matriz1[0])):
21         resultado[i][j] = matriz1[i][j] + matriz2[i][j]
22
23 # Mostrar resultado
24 print("Matriz 1:")
25 for fila in matriz1:
26     print(fila)
27
28 print("\nMatriz 2:")
29 for fila in matriz2:
30     print(fila)
31
32 print("\nSuma de matrices:")
33 for fila in resultado:
34     print(fila)
```

Listing 5: Suma de matrices

4.2. Ejemplo: Multiplicación de matrices

```
1 # Definimos matrices para multiplicación
2 A = [
3     [1, 2, 3],
```



```

4      [4, 5, 6]
5  ] # Matriz 2x3
6
7  B = [
8      [7, 8],
9      [9, 10],
10     [11, 12]
11 ] # Matriz 3x2
12
13 # Crear matriz resultado 2x2 inicializada con ceros
14 C = [[0, 0], [0, 0]]
15
16 # Multiplicación de matrices A*B
17 for i in range(len(A)):
18     for j in range(len(B[0])):
19         for k in range(len(B)):
20             C[i][j] += A[i][k] * B[k][j]
21
22 print("Matriz A (2x3):")
23 for fila in A:
24     print(fila)
25
26 print("\nMatriz B (3x2):")
27 for fila in B:
28     print(fila)
29
30 print("\nResultado A*B (2x2):")
31 for fila in C:
32     print(fila)

```

Listing 6: Multiplicación de matrices

5. Ciclo for con Tuplas

Las tuplas son estructuras de datos inmutables que pueden contener elementos de diferentes tipos. Son similares a las listas pero no pueden modificarse después de su creación.

5.1. Ejemplo: Recorriendo una tupla

```

1 # Definimos una tupla con información de un estudiante
2 estudiante = ("Ana García", 20, "Ingeniera", 9.5)
3
4 # Recorremos la tupla
5 print("Información del estudiante:")
6 for i, dato in enumerate(estudiante):
7     print(f"Campo {i+1}: {dato}")

```

Listing 7: Recorriendo elementos de una tupla

5.2. Ejemplo: Tupla de tuplas (matriz inmutable)

```

1 # Matriz como tupla de tuplas (inmutable)
2 matriz_tupla = (
3     (1, 2, 3),
4     (4, 5, 6),
5     (7, 8, 9)
6 )
7
8 print("Matriz como tupla:")
9 for fila in matriz_tupla:
10     for elemento in fila:
11         print(f"{elemento:2d}", end=" ")
12     print() # Salto de línea después de cada fila

```

Listing 8: Recorriendo una matriz representada con tuplas

5.3. Ejemplo: Desempaquetado en tuplas

```
1 # Lista de tuplas con coordenadas (x, y)
2 puntos = [(1, 2), (3, 4), (5, 6), (7, 8)]
3
4 print("Coordenadas de puntos:")
5 for x, y in puntos:
6     print(f"Punto: ({x}, {y}) - Distancia al origen: {(x**2 + y**2)**0.5:.2f}")
```

Listing 9: Desempaquetado de tuplas en ciclo for

6. Ejercicios Prácticos

A continuación se presentan algunos ejercicios para practicar el uso del ciclo for con diferentes estructuras de datos.

6.1. Ejercicio 1: Procesamiento de listas

```
1 # Dada una lista de n meros , crear una nueva lista solo con los n meros pares
2 numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
3 pares = []
4
5 for num in numeros:
6     if num % 2 == 0:
7         pares.append(num)
8
9 print(f"N meros originales: {numeros}")
10 print(f"N meros pares: {pares}")
```

Listing 10: Ejercicio: Filtrar números pares

6.2. Ejercicio 2: Diagonal de una matriz

```
1 # Extraer los elementos de la diagonal principal de una matriz
2 matriz = [
3     [1, 2, 3],
4     [4, 5, 6],
5     [7, 8, 9]
6 ]
7
8 diagonal = []
9 for i in range(len(matriz)):
10     diagonal.append(matriz[i][i])
11
12 print("Matriz:")
13 for fila in matriz:
14     print(fila)
15 print(f"\nDiagonal principal: {diagonal}")
```

Listing 11: Ejercicio: Extraer diagonal de una matriz

6.3. Ejercicio 3: Operaciones con tuplas

```
1 # Calcular estadísticas básicas de una tupla de n meros
2 datos = (15, 23, 18, 27, 32, 19, 22, 25)
3
4 # Calcular suma
5 suma = 0
6 for num in datos:
7     suma += num
8
9 # Calcular promedio
10 promedio = suma / len(datos)
11
12 # Encontrar máximo y mínimo
13 maximo = datos[0]
```

```

14 minimo = datos[0]
15 for num in datos:
16     if num > maximo:
17         maximo = num
18     if num < minimo:
19         minimo = num
20
21 print(f"Datos: {datos}")
22 print(f"Suma: {suma}")
23 print(f"Promedio: {promedio:.2f}")
24 print(f"Maximo: {maximo}")
25 print(f"Minimo: {minimo}")

```

Listing 12: Ejercicio: Estadísticas de tupla

7. Conclusión

El ciclo `for` es una herramienta poderosa y versátil en Python que permite trabajar eficientemente con diferentes tipos de estructuras de datos. Como hemos visto:

- Con **listas**, podemos modificar elementos y realizar operaciones complejas.
- Con **arreglos**, podemos trabajar con datos unidimensionales y bidimensionales.
- Con **matrices**, podemos realizar operaciones algebraicas como suma y multiplicación.
- Con **tuplas**, podemos procesar datos inmutables de manera segura.

La comprensión del ciclo `for` es fundamental para el desarrollo de algoritmos y el procesamiento de datos en Python.

```
# sumatoria de gauss
def suma_gauss(n):
    y = (n*(n+1))/2
    return y

suma_gauss(100)
```

5050.0

```
#ciclo for para la misma suma de gauss

s = 0
for a in range(1,101):
    s = s+a

print(s)
```

5050

```
x = 0
for i in range(1,101):
    x = x+i**2

print(x)
```

338350

```
def sumatoria_i_cuadrado(n):
    andres = (n*(n+1)*(2*n+1))/6
    return andres

sumatoria_i_cuadrado(100)
```

338350.0

```
#for

w = 0
for i in range(1,101):
    w = w+i**3

print(w)
```

25502500

```
#función

def suma_al_cubo(n):
    y = (n*(n+1)/2)**2
    return y

suma_al_cubo(100)
```

25502500.0

```
andres = list(range(1, 11))
print(andres)
```

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

```
# Definimos una lista de numeros
numeros = list(range(1,101))

# Variable para almacenar la suma
suma_total = 0

# Recorremos cada elemento de la lista
for numero in numeros :
    suma_total += numero
```

```
print(f'Sumando { numero }, suma actual : { suma_total }')  
  
print(f"La suma total es : { suma_total }")
```

```
Sumando 72 , suma actual : 2628  
La suma total es : 2628  
Sumando 73 , suma actual : 2701  
La suma total es : 2701  
Sumando 74 , suma actual : 2775  
La suma total es : 2775  
Sumando 75 , suma actual : 2850  
La suma total es : 2850  
Sumando 76 , suma actual : 2926  
La suma total es : 2926  
Sumando 77 , suma actual : 3003  
La suma total es : 3003  
Sumando 78 , suma actual : 3081  
La suma total es : 3081  
Sumando 79 , suma actual : 3160  
La suma total es : 3160  
Sumando 80 , suma actual : 3240  
La suma total es : 3240  
Sumando 81 , suma actual : 3321  
La suma total es : 3321  
Sumando 82 , suma actual : 3403  
La suma total es : 3403  
Sumando 83 , suma actual : 3486  
La suma total es : 3486  
Sumando 84 , suma actual : 3570  
La suma total es : 3570  
Sumando 85 , suma actual : 3655  
La suma total es : 3655  
Sumando 86 , suma actual : 3741  
La suma total es : 3741  
Sumando 87 , suma actual : 3828  
La suma total es : 3828  
Sumando 88 , suma actual : 3916  
La suma total es : 3916  
Sumando 89 , suma actual : 4005  
La suma total es : 4005  
Sumando 90 , suma actual : 4095  
La suma total es : 4095  
Sumando 91 , suma actual : 4186  
La suma total es : 4186  
Sumando 92 , suma actual : 4278  
La suma total es : 4278  
Sumando 93 , suma actual : 4371  
La suma total es : 4371  
Sumando 94 , suma actual : 4465  
La suma total es : 4465  
Sumando 95 , suma actual : 4560  
La suma total es : 4560  
Sumando 96 , suma actual : 4656  
La suma total es : 4656  
Sumando 97 , suma actual : 4753  
La suma total es : 4753  
Sumando 98 , suma actual : 4851  
La suma total es : 4851  
Sumando 99 , suma actual : 4950  
La suma total es : 4950  
Sumando 100 , suma actual : 5050  
La suma total es : 5050
```

Comienza a programar o [generar](#) con IA.

